

SEGMENT 10 • TOOLING AND CONTEXT

# Tools the agent finds. Context the assistant needs.

The Sarvam MCP server at runtime — and four ways to stop your coding assistant inventing APIs

MCP voice agents • Markdown • llms.txt • Context7 • Agent Skills • everything measured

---

Dr. Bhaveshkumar C. Dharmani • PhD (ICT), DA-IICT Gandhinagar • Founder, AI Vidhya4Sarvam

<https://www.aividhya.in/> • <https://www.aividhya4sarvam.in/> • [bhavesh@aividhya.in](mailto:bhavesh@aividhya.in) • <https://www.linkedin.com/in/bhaveshdharmani/>

# Two different problems, often confused

## 1 Runtime context — what the AGENT can reach

- Your deployed agent needs to transcribe, translate, speak
- MCP describes those capabilities in a standard protocol
- The model chooses which tool to call, and when
- Failure mode: it picks the wrong tool, on a live call
- You pay in milliseconds and in schema tokens, every turn
- Lab 11 measures all three costs

## 2 Build-time context — what YOU can reach

- Your coding assistant writes Sarvam code that does not run
- Markdown, llms.txt, Context7 and Agent Skills each fix part of it
- The model never calls anything — it just writes better code
- Failure mode: confident, plausible, non-existent methods
- You pay once, in setup, and the benefit compounds
- Lab 12 measures which layer actually helps

★ **Both are context engineering. One feeds a running agent, the other feeds the person building it. Confusing them is why teams adopt MCP and wonder why their code still does not compile.**



## MCP at runtime

Fourteen tools, zero wrappers — and the three prices nobody quotes

# The whole integration is a config block

ANY MCP CLIENT — CLAUDE CODE, CURSOR, WINDSURF, ZED, n8n

```
{
  "mcpServers": {
    "sarvam": {
      "command": "uvx", "args": ["sarvam-mcp"],
      "env": { "SARVAM_API_KEY": "<YOUR_KEY>" }
    }
  }
}
```

| Namespace      | Tools | What it is for                                                                                       | Costs ₹          |
|----------------|-------|------------------------------------------------------------------------------------------------------|------------------|
| sarvam_tools_* | 23    | STT · TTS · translate · transliterate · LID · LLM · vision · pronunciation · dub · localize · recall | Yes — real calls |
| sarvam_code_*  | 7     | api_reference · languages · speakers · pricing · snippet · recommend_model · validate_request        | Effectively free |



Give a production voice agent the runtime tools ONLY. Hand it the builder tools and you have invited it to read documentation in the middle of a phone call.

# The agent that writes no plumbing

## PYTHON • the entire voice agent

```
tools = await MultiServerMCPClient(SERVER).get_tools() # 30 tools
agent = create_agent(llm, tools=tools) # langchain.agents, not langgraph.prebuilt

await agent.ainvoke({"messages": [{"role": "user", "content":
    "Transcribe ./data/call.wav in Hindi, then summarise it in English"}]})

# → agent picks sarvam_stt_transcribe, calls it, reads the transcript,
# reasons over it, and answers. You wrote no wrapper. None.
```



### Compare with Lab 08

There, listen() / think() / speak() was about sixty lines you owned and maintained. Here it is three lines and a config block.



### So why would anyone still hand-wire it?

Because those sixty lines bought you something. The next three slides measure exactly what.

# Price one — the ₹ meter goes blind

✕

## What you expect

- Agent transcribes ten minutes of audio
- Agent speaks ten replies through Bulbul
- Your CostMeter reports what it cost
- You reconcile against the dashboard
- Everyone is happy

!

## What actually happens

- The SDK call happens INSIDE the MCP server
- Tool results carry no usage back through the adapter
- `response.usage` never reaches your notebook
- Your meter prints ₹0.00 — confidently
- The dashboard disagrees at the end of the month

★

**The general law: put a protocol boundary between your code and a billed API, and your observability stops at that boundary. Whatever you cannot see you cannot cost — so meter at the boundary you DO control.**

# Price two — and the measurement that refused to cooperate

WHAT LAB 10 ACTUALLY PRINTED — three runs each

```
direct SDK :    1611 ms  ['435', '805', '3593']  <- one 3.6 s outlier
via MCP      :     821 ms  ['934', '787', '743']  <- tight, consistent
MCP overhead: -790 ms per call      # i.e. MCP came out FASTER
```

1

**The expected answer was 'MCP is slower'. The data said otherwise**

Not because MCP is fast, but because  $n=3$  over a live network is dominated by variance. One 3.6-second outlier moved the direct-SDK mean by more than the protocol hop could.

★

**This is the most useful thing in the lab, and it is not about MCP**

A benchmark whose spread is larger than its effect has not measured the effect. Report the raw samples, not just the mean — the list of three numbers is what makes the problem visible.

→

**So decide on architecture, not on this number**

A stdio round-trip is real but small. What actually rules MCP out of the hot path is variance and an extra failure mode on a call a human is waiting on — not a mean you can measure in three

★

**Run it live and let the room watch the outlier wreck the mean. Teaching somebody to distrust their own three-sample benchmark is worth more than any number this slide could print.**

# Price three — you pay for tools you never call

8,518

tokens of tool schema  
injected on EVERY turn

₹4.99

per 20-turn call  
schemas alone

₹4.99L

per 100k calls  
before a word is spoken

₹4.63L

saved by trimming  
30 tools → 5



## Thirty tools is 8,518 tokens before your caller says a word

Injected on every turn, twenty times over a twenty-turn call. The single largest schema is sarvam\_tools\_stt\_transcribe at 661 tokens.



## A voice agent needs five tools, not thirty

STT, TTS, translate, transliterate, identify\_language. Filter by EXACT tool name — a substring match like "speak" also catches sarvam\_code\_speakers.



## And the filter usually raises accuracy too

Fewer options is a smaller decision for the model. Cheaper and more reliable at the same time — the rare trade that is not a trade.



₹4.6 lakh per 100k calls, recovered by deleting twenty-five tools the agent was never going to call. Nothing costs more than an option that only exists to be ignored.



# So where does MCP belong in a voice product?

## H Hot path → direct SDK

- The STT → LLM → TTS loop the caller is waiting on
- Every millisecond is audible to a human being
- Three calls you already know and will not change
- You want to see and tune every single hop
- High volume, where schema tokens compound
- This is Lab 08, and it stays that way

## A Around it → MCP

- Post-call summarisation and translation of transcripts
- Document lookups and pronunciation-dictionary updates
- Analytics, enrichment, anything asynchronous
- Capability you want as configuration, not code
- Tool sets shared across several agents or products
- This is Lab 11, and it is genuinely better here

★ This is the same build-versus-platform line from Segment 05, drawn one level lower — and now you can draw it with numbers you measured rather than numbers a vendor gave you.

B

# Context engineering

Four ways to stop your coding assistant inventing APIs that do not exist

# The failure every team hits in week one

## WHAT AN ASSISTANT WITH NO CONTEXT WRITES

```
resp = client.chat.completions.create(          # ← AttributeError. No .create
    messages=[{"role": "user", "content": text}] # ← no model= : defaults drift
)
r = client.speech_to_text.transcribe(file=f, sample_rate=8000) # ← TypeError
```

1

### It is not being stupid — it is pattern-matching

It has read a hundred thousand OpenAI examples and close to zero Sarvam ones. So it writes the shape it knows. Every line above is on your gotchas list already.

2

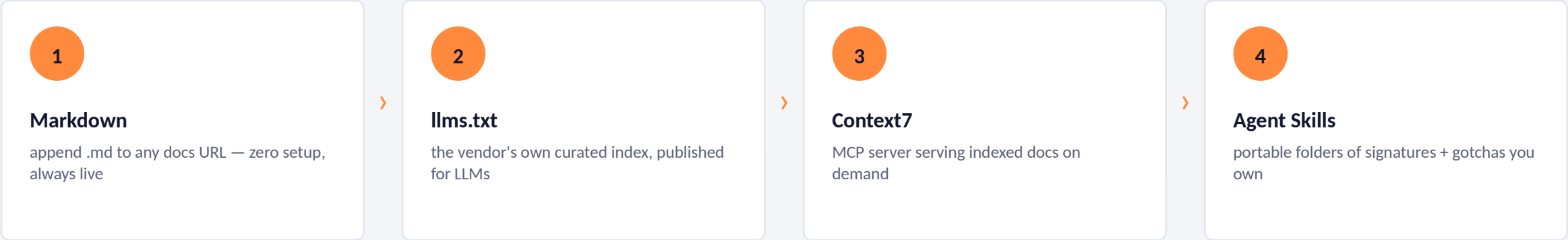
### A bigger model does not fix this

This is a knowledge gap, not a reasoning gap. The fix is not a better model — the fix is better context. Four layers of it, and they cost between nothing and one config block.



Every wrong line above is already documented on the gotchas slide. The question is not whether you know the fix — it is whether your tooling knows it.

# The four layers, cheapest first



| Layer        | Setup            | Freshness  | Best for                              |
|--------------|------------------|------------|---------------------------------------|
| Markdown     | Zero             | Live       | One page, one question, right now     |
| llms.txt     | Zero             | Live       | "What does this platform even offer?" |
| Context7     | One config block | Index lag  | Working across many libraries at once |
| Agent Skills | One install      | You own it | Your team repeats the same mistakes   |

★

Docs **DESCRIBE** what the API offers. Skills **PRESCRIBE** what your team must do. You need both halves — and most teams only ever install the first.

# Two facts worth correcting out loud

1

## llms-full.txt is NOT part of the llms.txt standard

The spec defines exactly one file: an H1 (the only required part), an optional blockquote summary, and optional H2 sections of links. `llms-full.txt` is a widely-adopted convention for "everything, concatenated" — Sarvam ships one — but do not expect every vendor to.

2

## Sarvam is already indexed in Context7 — library ID /websites/sarvam\_ai

No API key needed, docs pre-indexed. But say the caveat too: Context7 re-crawls on its own schedule, so for an API shipping this fast the vendor's own llms.txt is fresher by definition.



## And one correction to my own gotchas slide

The 8 kHz row used to say "no sample\_rate → garbage transcript". Lab 02 disproved it: sample\_rate is not a REST parameter at all. It belongs to the streaming APIs, where raw PCM has no header. A rubric that encodes a stale belief punishes correct code.

# An Agent Skill, in one slide

```
skills/sarvam-cost-metering/SKILL.md
```

```
---  
name: sarvam-cost-metering          # <=64 chars, a-z0-9-, MUST match the folder  
description: Adds rupee cost tracking to Sarvam AI API calls. Use whenever  
    writing or reviewing Python that calls the sarvamai SDK...  # <=1024 chars  
license: Apache-2.0  
---  
  
# Sarvam cost metering  
Every Sarvam API call costs money. Attach a meter entry to every billed call...
```

1

## Progressive disclosure is the design constraint

name + description (~100 tokens) load for EVERY installed skill at startup. The body loads only on activation — keep it under 500 lines / 5,000 tokens. Detail goes in references/, loaded only when needed.

2

## The description is doing triage, not marketing

It is read by a model that has not opened your skill yet, deciding whether to. Say what it does AND when to use it. Write it for that job.

# Does any of it actually work? Measure it.



## Your gotchas slide becomes an automated grader — now 14 rules, not 8

One regex per gotcha, matching the WRONG pattern. The first 8-rule version scored a context-free generation 8/8 while the code carried six real defects none of the rules described. An incomplete rubric does not merely miss mistakes — it manufactures confidence. Grow it every time you find a trap.

| Context layer          | Score      | Context tokens | What it can and cannot fix                                              |
|------------------------|------------|----------------|-------------------------------------------------------------------------|
| 0 • no context         | measure it | 0              | the baseline you must beat                                              |
| 1 • markdown page      | measure it | ~2–4k          | only mistakes THAT page documents — a chat page cannot fix an STT error |
| 2 • llms.txt retrieval | measure it | ~4–8k          | bounded by whether keyword scoring found the right pages                |
| 3 • agent skill        | measure it | ~0.5k          | smallest payload; covers exactly what you wrote into it                 |
| 4 • skill + markdown   | measure it | ~6–12k         | usually the winner — prescription plus description                      |



Do not expect a clean sweep from every layer, and do not read a low score as "the model is bad at coding" — a doc page can only fix what it documents. Only a skill written AGAINST the rubric should be expected to clear it.

# The recipe card the cook actually keeps

*Every professional kitchen owns the same shelf of cookbooks. Nobody reads them during service.*

*What they read is the card taped inside the cupboard door — three lines in the head chef's handwriting.  
"Our stock is salted. Do not season before reducing. The oven runs hot by 15 degrees."*

*That card is not better than the cookbooks. It is smaller, it is specific to this kitchen, and it is where the burnt dinners are recorded.*



Documentation tells your assistant what the API offers. A skill tells it what YOUR team gets wrong. Add one line every time somebody hits a trap, and in six months that file is the most valuable engineering artefact you own.



# Take this home

## **S** An Agent Skill — prescriptive, yours

- `npx skills add sarvamai/skills`
- Then write your own for your house rules
- SDK signatures and the traps, not the tour
- Under 500 lines; detail in references/
- Grows one line at a time, forever
- Loaded automatically by every assistant your team uses

## **L** Plus `llms.txt` — descriptive, current

- `docs.sarvam.ai/llms.txt` — the map
- `docs.sarvam.ai/llms-full.txt` — the territory
- Append `.md` to any page for a clean version
- Nothing is fresher; the vendor publishes it
- Costs nothing and needs no setup at all
- Context7 on top if you work across many libraries

★ One layer prescribes, the other describes. Together they cover both halves of the problem — and the whole setup is an afternoon, once.

“

The model was never the bottleneck.  
What it could reach was.

— End of Segment 10. Labs 11 and 12 are the hands-on: measure all of it yourself.